

Cloud Computing Lab 1: Introduction to APIs and FastAPI

Due Date

Monday, September 8, 2025 11:59 PM

Objective

In this lab, you will learn the basics of what an API is, how to create one using FastAPI, and how to serve HTML responses using templates.

What is an API?

An Application Programming Interface (API) allows different systems to communicate with each other. APIs provide endpoints which can be used to create, retrieve, update, or delete (CRUD) data in a structured way.

Setup

Before starting, make sure you have Python installed. If not, download it from <https://www.python.org/downloads/>. You'll also need to install FastAPI and uvicorn, which can be done using pip in your terminal:

```
1 > pip install fastapi uvicorn
```

Creating Your First FastAPI Endpoint:

Follow these steps to create a “Hello World” endpoint:

1. Create a new Python file, e.g., `main.py`.
2. Add the following code:

```
from fastapi import FastAPI

app = FastAPI()

@app.get("/")
def read_root():
    return {"message": "Welcome to this fantastic app!"}
```

This code defines an endpoint (the root endpoint, “/”, see **line 5**) and has it return data.

3. Run the application by going to your terminal and using the command:

```
1 > uvicorn main:app --reload
```

4. Your terminal should have told you uvicorn is now running on something like `http:127.0.0.1:8000`. Go to that link and you should see your JSON response. That's because the default URL is considered the root endpoint.

Reading in Data from a File

Obviously, we're not going to be just display a basic message from this API. Instead, you're going to be displaying data that we'll give you. To demonstrate how to set up more endpoints, with specific names, and to read in data from a file, we'll follow these steps:

1. Create a new JSON file (like `users.json`)
2. Copy the following JSON data and paste it there:

```
{
  "users": [
    {
      "name": "John Doe",
      "phone": 1234567890,
      "fave_color": "red"
    },
    {
      "name": "Jane Doe",
      "phone": 9987654321,
      "fave_color": "blue"
    },
    {
      "name": "The Guy",
      "phone": 5555555555,
      "fave_color": "silver"
    }
  ]
}
```

3. Remember to import the `json` module at the top of your Python file before proceeding. Now, we'll update the Python code to include a function with a new endpoint called `users`:

```
@app.get("/users")
def get_users():
    with open("users.json", "r") as file:
        # Read in files per line, joining them by an empty string
        file_contents = "".join(file.readlines())

        # Convert the string contents to json
        data = json.loads(file_contents)

        # Return the resulting JSON data
        return data
```

4. Now, starting up your uvicorn server and navigating to `http://127.0.0.1:8000/users` should display the user data that was in our file.

Serving HTML with FastAPI and Templates

None of what we've just made is pretty at all, even if it is functional. So, let's figure out how to serve some HTML to make our data more presentable.

1. Install Jinja2 for creating HTML templates:

```
1 pip install jinja2
```

2. Create a `templates` directory in the same location as your Python script.

3. Inside the `templates` directory, create an HTML file `index.html`:

```
<!DOCTYPE html>
<html>
<head>
    <title>My First HTML Endpoint</title>
</head>
<body>
    <h1>{{ message }}</h1>
</body>
</html>
```

4. Modify `main.py` to use templates:

```
from fastapi import FastAPI, Request
from fastapi.templating import Jinja2Templates

app = FastAPI()
templates = Jinja2Templates(directory="templates")

@app.get("/")
def read_root(request: Request):
    return templates.TemplateResponse("index.html", {"request": request, "message": "Hello World from FastAPI!"})
```

This allows FastAPI to use an HTML file as a template on how to format the data. In this case, the data is:

```
1 {
2     "message": "Hello, World from FastAPI!"
3 }
```

If you wanted to, you could also update your `users` endpoint to be formatted to a template. However, since the data there has an array of JSON objects, it won't be as simple. If you'd like to practice with that sample data above, since it'll be similar to your assignment for this lab, feel free to use the Jinja Template Documentation

5. Visit `http://127.0.0.1:8000/`. You should now see an HTML page with a greeting.

DIY

For this lab, what you'll submit is a list of movies. This list should be based on the data we give you (`movies.json`). The data is grabbed from an API itself, TMDB is a movie API, and the file we supply consists of popular movies from when the file was created. Using a new endpoint, your API should display every movie in an HTML template, and for each movie should show at least:

- The Title
- The Poster

NOTE: For this, you'll need to use TMDb API to get the pictures. The poster path is given to you in the data, and is used in combination with the link <https://image.tmdb.org/t/p/original> to get the image. For example, the poster path for the hit movie Spider-Man 2 is `/olxpyq9kJAZ2NU1siLshhhXEPR7.jpg`, making the full link <https://image.tmdb.org/t/p/original/olxpyq9kJAZ2NU1siLshhhXEPR7.jpg>. You'll need to figure out how to do this in your template though :)
- The Release Date of the Movie

Outside of this, you can display any of the other information for each movie if you want to. My graders should be able to clone your repo and run this server locally and see the result of your work!

One other thing we'd like to see in this one is styling. Look up some CSS tutorials to add some sort of creative flair to it. This could be putting some cool borders around the posters to make them look like art installations, or some animations that when you hover over a title it shows more information. Something to make it uniquely yours.

Tips and Reminders:

- If you ever get an error running your uvicorn server, such as the command doesn't exist, try running it through python:

```
1 > py -m uvicorn main:app --reload
```

for Windows, or

```
1 > python -m uvicorn main:app --reload
```

for MacOS.

- Remember to run your application in a virtual environment to avoid dependency conflicts.
- Use the `--reload` flag with uvicorn for automatic reloading during development.
- Ensure your HTML files are in the correct directory as specified in your Python script.
- If you're having trouble with the templates, consult the Jinja Template Documentation

Grading

- 5 pts - Create FastAPI Endpoint running locally
- 5 pts - Read Data from `users.json`
- 5 pts - Serve Formatted `users.json` data using Jinja2Templates
- 20 pts - Serve movies data using Jinja2Templates

Submission

Submit this assignment to your lab 1 repo in GitHub before the due date.

What to Expect Next Lab

In the next lab, we will expand on what we've learned by interacting with cloud storage services like AWS S3.